

CCF2026开源创新大赛

Mooncake 赛题2初赛技术报告

Mooncake Store 碎片感知分配优化

Fragmentation-Aware Allocation for Mooncake Store

参赛方向	赛题2：优化Mooncake Store吞吐性能、高可用功能和可扩展性，优化SGLang HiCache+Mooncake Store 性能
作品名称	Mooncake Store 碎片感知分配策略
参赛仓库	https://github.com/Lorry1024/ccf2026-mooncake-fragmentation-aware
上游PR	https://github.com/kvcache-ai/Mooncake/pull/2797
PR头提交	0123fa1 Fix fragmentation-aware allocation test setup
报告日期	2026年7月9日

本报告与仓库中的patch、复现实验、验证日志和展示PPT共同构成初赛提交材料。

摘要

Mooncake Store 是 Mooncake 体系中支撑 KVCache 对象存储和复用的关键组件。现有 `free_ratio_first` 策略根据 `segment` 总空闲比例进行排序，能够改善整体空间利用均衡，但在混合大小 KVCache 对象和长时间运行的场景下，总空闲空间可能被切分为多个小空洞，无法真实反映当前大对象是否可以被连续放入。本文提出并实现 `fragmentation_aware` 策略，在保持默认行为不变和有界候选采样不变的前提下，将最大连续空闲区域、连续性比例和当前请求可容纳性纳入排序逻辑，使 Master 优先选择能够直接满足请求的 `segment`。

本作品已形成 Mooncake 上游 draft PR，当前 PR 头提交 `0123fa1` 已通过 GitHub Actions，结果为 26 个检查成功、1 个检查跳过。确定性专题仿真显示，在 6 个大对象请求上，`free_ratio_first` 的首选 `segment` 可直接容纳次数为 0/6，而 `fragmentation_aware` 为 6/6；`fallback` 尝试次数从 11 降至 0；平均候选 `segment` 数量保持 5.00；本地单次决策耗时从 121.00ns 增加到 138.93ns，额外开销约 17.93ns。该结果说明，新策略能够以很小的决策成本提升碎片化场景下的分配稳定性。

关键词： Mooncake Store；KVCache；碎片感知；连续空闲区域；分配策略；SGLang HiCache

模块完成情况

表 1: 初赛阶段模块完成情况

模块	完成情况
问题分析	明确 <code>free_ratio_first</code> 在碎片化大对象分配中的误判模式，形成最小复现和专题仿真。
策略设计	设计 <code>fragmentation_aware</code> 排序规则，优先考虑 <code>can_fit</code> 、连续性评分和最大连续空闲区域。
代码实现	在 Mooncake Store 中新增策略类型、策略类、配置解析、启动参数和 <code>benchmark</code> 矩阵。
测试验证	新增碎片化单元测试，保留 <code>preferred segment</code> 语义，PR 通过上游 GitHub Actions。
材料整理	完成中文 Markdown 材料、LaTeX 技术报告、展示 PPT、复现实验源码和提交包。
待完成项	5 分钟展示视频尚未录制；真实 RDMA 和 SGLang HiCache 端到端 <code>benchmark</code> 作为后续工作。

目录

1	赛题理解与项目定位	1
2	原有机制与问题分析	1
2.1	Mooncake Store 分配链路 with 关键抽象	1
2.2	现有策略的能力边界	2
2.3	总空闲比例误判的形成机制	3
2.4	误判带来的性能影响	3
2.5	本项目抽象出的优化需求	3
3	碎片感知分配策略设计	4
3.1	设计目标 with 非目标	4
3.2	候选元数据 with 评分字段	4
3.3	排序规则	6
3.4	完整策略流程	7
3.5	复杂度 with 并发安全分析	7
4	实现路径	8
4.1	代码改动总览	8
4.2	策略类接入	8
4.3	配置 and 工厂接入	10
4.4	单元测试设计	10
4.5	测试修复 with CI 闭环	10
4.6	文档、benchmark with 复现材料	11
5	验证与效果	12
5.1	CI 验证	12
5.2	专题仿真指标	12
5.3	结果解释	13
6	与赛题要求的对应	13
7	使用与复现	14

7.1	启用策略	14
7.2	应用补丁	14
7.3	回滚补丁	14
7.4	复现实验	14
8	边界与后续工作	15
9	结论	15
A	提交材料清单	15
B	参考链接	16

1 赛题理解与项目定位

Mooncake 赛题2要求围绕Mooncake Store吞吐性能、高可用功能、可扩展性以及SGLang HiCache+Mooncake Store性能开展优化。该要求并不只局限于网络传输或外层推理框架，也包含Store后端内部影响吞吐稳定性和资源利用率的关键路径。对KV-Cache场景而言，Store中对象的分配位置会影响一次写入是否能顺利落地，也会影响后续fallback、重试、淘汰和副本放置压力。

本项目选择的切入点是Mooncake Store分配策略层。该方向具有三个特点：第一，它位于真实Mooncake Store代码路径中，能够通过PR形式提交给上游；第二，它对SGLang HiCache等前端具有后端相关性，因为这些前端将KVCache对象交给Store时需要依赖Store的放置决策；第三，它可以在不重写协议、不引入重依赖、不改变默认行为的条件下完成可验证优化。

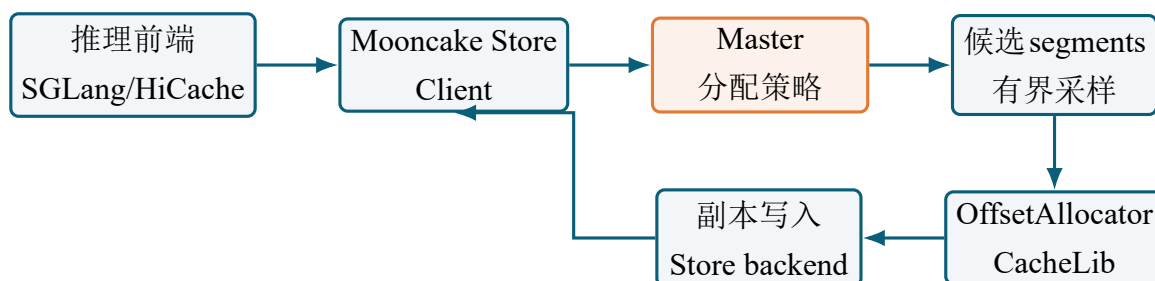


图 1: 本项目在Mooncake Store路径中的位置

2 原有机制与问题分析

2.1 Mooncake Store分配链路与关键抽象

Mooncake Store在写入KVCache对象时，需要为对象的每个副本选择可写入的segment。上层调用并不直接关心底层空闲块的形态，而是把对象大小、副本数、preferred segment和excluded segment等约束交给Master侧分配逻辑。Master再调用具体分配策略，从可用segment集合中选出候选，并通过对应allocator完成空间申请。这个过程看似只是“选择一个segment”，但实际承担了三类语义：

1. **容量语义**。当前segment是否拥有足够空间，决定对象是否可以一次性落地。
2. **副本语义**。多副本写入需要避免重复使用同一segment，并在候选不足时保留best-effort fallback。
3. **调度语义**。preferred segment表示上层希望优先使用的放置位置，excluded segment表示本次请求不能选择的位置。

因此，分配策略不是孤立的性能小优化，而是Store吞吐稳定性、资源利用率和高可用副本放置之间的连接点。如果策略选中了“总空闲很多但无法容纳当前对象”的segment，后续allocator仍会拒绝分配，系统只能继续尝试其他segment或走fallback路径。该失败不会直接制造数据错误，但会增加一次无效尝试，并可能放大为锁竞争、重试、尾延迟和副本放置压力。

表 2: Mooncake Store 分配路径中的关键对象

对象	主要职责	与本项目的关系
Master	管理segment视图并执行分配策略	新策略接入点位于Master侧策略工厂和配置解析路径
segment	Store后端的空间放置单元	每个segment的空闲空间形态决定当前对象是否能落地
allocator	维护segment内部可用空间	提供总空闲空间和最大连续空闲区域等统计信息
preferred segments	上层传入的优先候选位置	新策略必须先尝试preferred，不能因为评分逻辑改变用户语义
excluded segments	当前请求禁止选择的位置	新策略在采样和fallback阶段都必须尊重排除集合

2.2 现有策略的能力边界

Mooncake Store已有多种分配策略。默认random策略开销低、行为简单，适合通用场景；free_ratio_first策略按照总空闲比例排序，目标是让更空闲的segment优先吸收写入，从而改善整体空间利用均衡；ssd_free_ratio_first、cxl和local_first等策略则面向特定介质或拓扑。就赛题2而言，free_ratio_first是最重要的对比基线，因为它已经使用了空间信息，却仍然没有识别segment内部碎片。

free_ratio_first的核心假设可以概括为：总空闲比例越高，当前请求越可能分配成功。该假设在对象大小比较均匀、释放行为比较规则、segment内部空闲区较少被切碎时通常成立。但KVCache Store的运行状态并不总是满足这些条件。长上下文推理、不同batch形态、前缀复用和缓存淘汰会让对象大小差异扩大；对象生命周期不同步又会让释放位置分散。最终，一个segment可能拥有较高总空闲字节数，但这些空间被拆成多个小洞，无法提供一个足够大的连续区域。

2.3 总空闲比例误判的形成机制

在 allocator 需要连续空间的前提下，“总空闲空间”和“最大连续空闲区域”是两个不同指标。前者衡量还有多少空间未被使用，后者衡量当前请求能否被一次性放入。碎片化场景下，两者可能明显背离。图2展示了典型情况：上方 segment 的绿色空闲块数量多、总量不低，但每个块都较小；下方 segment 的总空闲量略低，却拥有足够大的连续区域。

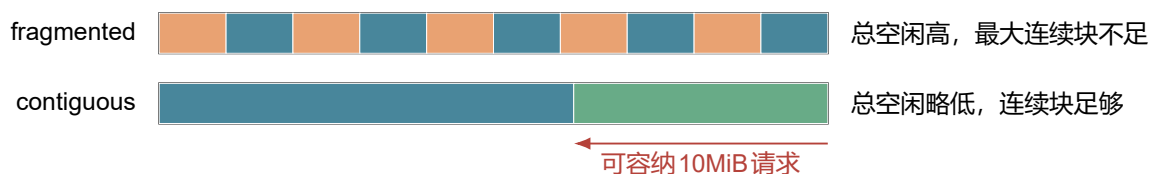


图 2: 总空闲空间与最大连续空闲区域可能不一致

最小复现数据如表3所示。`free_ratio_first` 会优先选择总空闲空间为 16MiB 的 `fragmented`，因为它的总空闲比例更高；但这个 segment 的最大连续空闲区域只有 8MiB，无法容纳 10MiB 请求。相反，`contiguous` 虽然总空闲空间只有 12MiB，却可以直接完成本次分配。

表 3: 最小碎片化复现场景

segment	总容量	总空闲	最大连续空闲	10MiB 请求
fragmented	32MiB	16MiB	8MiB	不能容纳
contiguous	32MiB	12MiB	12MiB	可以容纳

2.4 误判带来的性能影响

该问题的关键不在于最终一定失败，而在于首选候选质量下降。对多副本写入而言，一次错误排序可能导致一个副本先失败再尝试其他 segment；如果多个副本同时处于碎片化状态，失败尝试会被放大。对推理系统而言，这类失败尝试会影响 Store 写入路径的稳定性，进而影响 KVCache 对象的后端落盘、复用和淘汰压力。

2.5 本项目抽象出的优化需求

基于上述分析，本项目没有选择重写 Store 协议或引入复杂全局整理机制，而是把问题收敛为“在现有候选集合内提高排序质量”。这个抽象有明确边界：它不尝试移动已有对象，不做在线压缩，不改变副本协议，也不扩大候选扫描范围；它只利用 allocator

表 4: 总空闲比例误判的影响链路

阶段	表现	对系统的影响
候选排序	高总空闲但碎片化的 segment 排在前面	首选 segment 质量下降
allocator 申请	最大连续区域小于请求大小	出现一次无效分配尝试
副本放置	当前副本继续尝试其他 segment	增加 fallback 和重试路径压力
长时间运行	释放和再分配持续制造小空洞	问题随对象大小混合程度和运行时长加重
推理场景	KVCache 对象写入路径不稳定	间接影响 HiCache 后端稳定性和尾延迟

已经能提供或可近似得到的元数据，让策略在面对大对象请求时先识别“当前是否真的能放下”。因此，优化需求可以拆成四点：

1. 在排序中显式加入最大连续空闲区域，而不是只看总空闲比例。
2. 在当前请求大小已知的情况下，优先选择 `can_fit=true` 的候选。
3. 保持原有采样规模和 fallback 语义，避免策略本身成为新的扩展性瓶颈。
4. 保持配置可选和默认行为不变，便于灰度、回滚和上游接受。

3 碎片感知分配策略设计

3.1 设计目标与非目标

`fragmentation_aware` 的目标是提高碎片化场景下的首选 segment 可用性。它不是一个通用碎片整理器，也不是面向所有场景的强制默认策略。设计时刻意区分目标和非目标，是为了避免优化范围失控。

3.2 候选元数据与评分字段

策略对每个采样候选 segment 计算五类字段。这里的关键是把“容量总量”和“空间形态”拆开：`free_ratio` 继续描述整体空闲程度，`largest_free_region` 和 `contiguity_ratio` 描述空闲区域是否集中，`can_fit` 则把当前请求大小纳入判断。

表 5: fragmentation_aware 的设计边界

类型	说明
目标1	在混合大小对象场景中，优先选择最大连续空闲区域能够容纳当前请求的segment。
目标2	沿用free_ratio_first的有界候选思想，只改变候选排序信号，不扩大搜索范围。
目标3	保留preferred segment、excluded segment、多副本去重和best-effort fallback 语义。
目标4	作为显式可选策略接入，不改变random默认行为。
非目标1	不进行对象迁移、在线压缩或全局碎片整理。
非目标2	不修改Mooncake Store 客户端协议、metadata 协议或后端存储格式。
非目标3	不宣称替代真实RDMA 集群和SGLang HiCache端到端压测。

表 6: 候选segment评分字段

字段	计算方式	设计含义
total_free	所有allocator可用空间之和	描述segment剩余空间总量
largest_free_region	allocator报告的最大连续空闲区域最大值	描述当前对象是否存在可落入的连续块
free_ratio	$\text{total_free} / \text{total_capacity}$	保留原有空间均衡信号
contiguity_ratio	$\text{largest_free_region} / \text{total_free}$	衡量空闲空间是否集中
can_fit	$\text{largest_free_region} \geq \text{request_size}$	直接判断当前请求是否可被候选容纳

在实现中，若某类 `allocator` 无法提供精确的最大连续空闲区域，策略退化为使用可用空闲空间作为估计值。这样做的原因是兼容性优先：新策略不能要求所有 `allocator` 一次性改造完成。对能提供精确碎片信息的 `OffsetAllocator`，策略会使用真实 `getLargestFreeRegion()`；对无法提供精确值的场景，行为退化为接近总空闲视角，不会阻断已有部署。

3.3 排序规则

评分公式如下：

$$\text{free_ratio} = \frac{\text{total_free}}{\text{total_capacity}} \quad (1)$$

$$\text{contiguity_ratio} = \frac{\text{largest_free_region}}{\text{total_free}} \quad (2)$$

$$\text{score} = 0.70 \times \text{contiguity_ratio} + 0.30 \times \text{free_ratio} \quad (3)$$

其中 0.70 和 0.30 的权重体现了本策略的取舍：在碎片化优化场景下，空闲空间是否集中比总空闲比例更关键；但完全忽略总空闲比例也不合理，因为当多个候选都能容纳当前请求时，更高总空闲比例有助于维持 `segment` 之间的负载均衡。因此排序采用分层规则，而不是只用单一 `score`：

1. `can_fit=true` 的候选优先于 `can_fit=false` 的候选。
2. 同为可容纳或同为不可容纳时，`score` 高者优先。
3. `score` 相近时，`largest_free_region` 大者优先。
4. 仍无法区分时，`free_ratio` 高者优先。
5. 最后使用稳定 `tie-break`，避免同一输入下排序非确定。

Listing 1: `fragmentation_aware` 候选排序伪代码

```
for candidate in sampled_segments:
    total_free = sum_free_bytes(candidate)
    total_capacity = sum_capacity(candidate)
    largest = largest_contiguous_free_region(candidate)
    free_ratio = total_free / total_capacity
    contiguity = largest / total_free
    can_fit = largest >= request_size
    score = 0.70 * contiguity + 0.30 * free_ratio

sort candidates by:
```

```

can_fit desc,
score desc,
largest desc,
free_ratio desc,
stable_index asc

```

3.4 完整策略流程

图3展示了 `fragmentation_aware` 的完整流程。这里需要强调两点。第一，`preferred segment` 阶段不进入评分排序，而是沿用原有优先尝试语义；这是为了保证调用方显式指定的位置仍具有最高优先级。第二，评分排序只发生在采样候选上，且排序后仍逐个调用原有 `allocator` 尝试分配；如果候选不足以满足副本数，策略继续进入原有 `fallback` 路径。

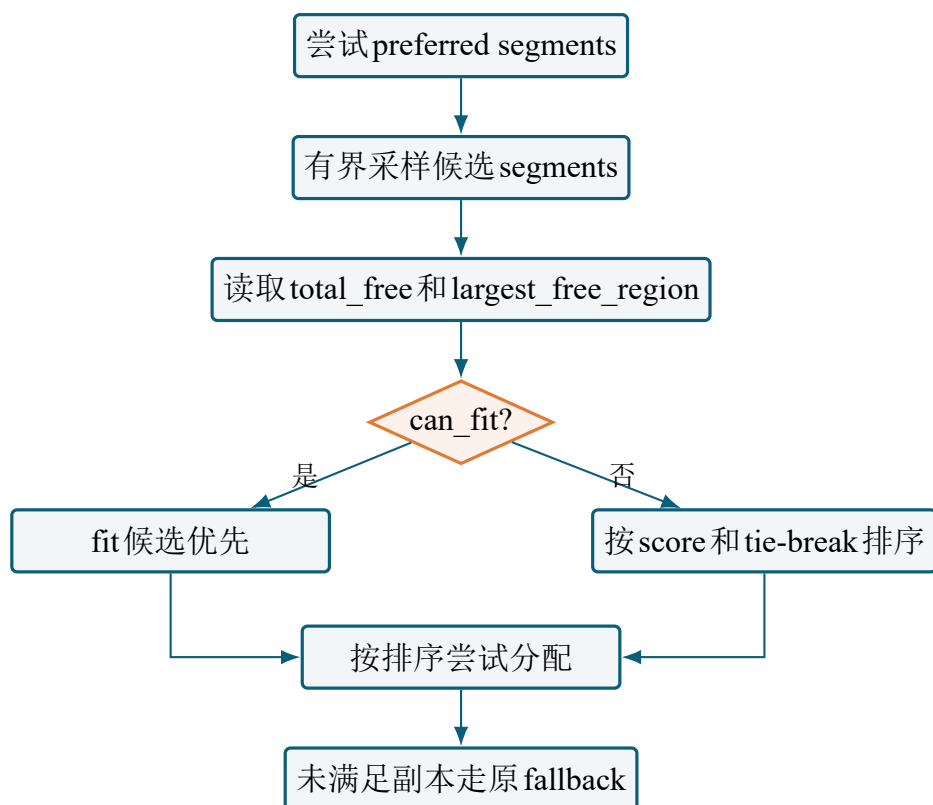


图 3: `fragmentation_aware` 策略流程

3.5 复杂度与并发安全分析

`fragmentation_aware` 没有引入新的全局数据结构，也没有在策略内部缓存跨请求状态。它只在每次分配时对采样候选读取 `allocator` 元数据，并对候选数组排序。若候

选数量为 k ，则额外计算复杂度为 $O(k)$ ，排序复杂度为 $O(k \log k)$ 。由于 k 来自现有有界采样，而不是全量segment数 N ，策略开销不会随集群规模线性扫描所有segment。

并发方面，新策略不改变allocator的实际申请路径，也不绕过已有锁和状态检查。评分阶段读取的空闲信息只能作为排序依据，不能作为最终成功承诺；真正分配仍由allocator执行。因此，即使评分后其他线程改变了某个segment的空闲状态，最坏结果也只是该候选分配失败并继续尝试后续候选，不会破坏空间一致性。

表 7: 策略复杂度与风险控制

维度	处理方式	风险控制
候选规模	沿用有界采样	避免全局扫描导致扩展性退化
元数据读取	读取总空闲和最大连续区域	只用于排序，最终结果由allocator确认
排序成本	对候选集合排序	候选数量小，本地仿真额外开销约17.93ns
并发变化	分配前后状态可能变化	保留原有失败重试和fallback路径
兼容性	无精确碎片信息时退化处理	不强制改造所有allocator

4 实现路径

4.1 代码改动总览

当前PR补丁涉及9个文件，覆盖策略实现、类型系统、配置解析、启动参数、测试、benchmark、文档和CI辅助步骤。表8列出主要改动。

4.2 策略类接入

核心实现位于mooncake-store/include/allocation_strategy.h。新类FragmentationAwareAllocationStrategy继承现有随机策略基础能力，而不是重写所有分配流程。这样做有两个好处：第一，可以复用原有单segment分配、随机fallback和排除集合处理；第二，可以把新增逻辑限制在候选评分和排序层，降低对主路径的侵入。

实现上，新策略先按原有语义处理preferred segment。如果preferred segment没有被排除、没有被当前请求重复使用，并且allocator能够完成申请，则直接返回该分配结

表 8: 关键代码改动

文件	作用
mooncake-store/include allocation_strategy.h	新增 FragmentationAwareAllocationStrategy , 实现候选评分、排序、preferred 处理和 fallback 衔接。
mooncake-store/include types.h	新增 FRAGMENTATION_AWARE 枚举值, 让策略进入统一类型系统。
mooncake-store/include master_config.h	解析 allocation_strategy=fragmentation_aware , 非法值继续走原有报错路径。
mooncake-store/src master.cpp	更新启动参数帮助信息, 确保用户可以从命令行发现新策略。
mooncake-store/tests allocation_strategy_test.cpp	增加碎片化定向测试、preferred segment 测试和参数化覆盖。
mooncake-store/benchmarks allocation_strategy_bench.cpp	将新策略加入分配策略 benchmark 矩阵, 便于后续真实环境对比。
docs/source/design mooncake-store.md	增加策略说明、适用场景和与其他策略的选择建议。
docs/source/deployment mooncake-store-deployment-guide.md	增加部署参数说明, 给出启用方式。
.github/workflows ci.yml	在 Go store binding 集成测试前释放 runner 磁盘空间, 避免无关环境失败。

果。只有在 `preferred` 阶段无法满足全部副本时，策略才进入候选采样和碎片感知排序。排序后的候选按顺序尝试分配，成功后加入 `used set`，避免同一次多副本请求重复选择同一 `segment`。

4.3 配置和工厂接入

为了让策略成为正式可选项，而不是测试代码中的孤立类，PR 在类型、配置和工厂三处同时接入：

1. 在 `AllocationStrategyType` 中新增 `FRAGMENTATION_AWARE` 枚举值。
2. 在 `master_config.h` 中识别字符串 `fragmentation_aware`。
3. 在策略工厂中为 `FRAGMENTATION_AWARE` 返回新策略实例。
4. 在 `master.cpp` 和部署文档中更新 `--allocation_strategy` 可选值。

新策略通过 Master 启动参数显式启用：

```
mooncake_master --allocation_strategy=fragmentation_aware
```

由于默认策略保持为 `random`，现有部署不会因为合入该 PR 而改变行为。若线上灰度时发现新策略不适合某类负载，只需要切回 `random` 或 `free_ratio_first` 即可回滚。

4.4 单元测试设计

测试不是简单地检查“新策略能运行”，而是针对问题定义构造确定性场景。核心测试创建两个 `segment`： `fragmented` 和 `contiguous`。前者总空闲空间更高，但最大连续区域小于请求大小；后者总空闲空间较低，但最大连续区域可以容纳请求。断言目标是： `fragmentation_aware` 应该选择 `contiguous`，而不是被更高总空闲比例误导。

另一个关键测试是 `preferred segment` 语义。即使某个 `preferred segment` 在评分上不是最优，只要它没有被排除且可以满足请求，新策略仍应优先选择它。这一点对兼容性很重要，因为 `preferred segment` 通常来自上层调度或副本放置约束，不能被内部评分逻辑无条件覆盖。

4.5 测试修复与CI闭环

PR 创建后经历了三类 CI 问题并逐步修复。第一次失败来自代码格式检查，后续通过格式化修复。第二次失败来自 GitHub runner 磁盘不足，Go store binding 集成测试在链接阶段出现空间不足；这不是策略逻辑失败，但会阻断 PR 验证，因此在对应 job 前

表 9: 关键测试覆盖点

测试点	构造方式	验证目的
碎片化选择	构造高总空闲但最大连续区域不足的 segment	验证策略不会只按总空闲比例排序
连续空间优先	构造总空闲略低但连续空间足够的 segment	验证 <code>can_fit</code> 优先级生效
preferred 保留	传入可用 preferred segment	验证用户显式优先级不被评分覆盖
参数化覆盖	将新策略加入既有策略测试矩阵	验证基础分配语义与已有策略一致
benchmark 接入	加入分配策略 benchmark 列表	为后续真实性能对比提供入口

增加了磁盘清理步骤。第三次失败来自最初碎片化测试构造不够严格：释放块与尾部空闲区域发生合并，导致最大连续区域大于预期，断言不稳定。后续测试改为构造 5 个 8MiB 块并释放内部块，避免尾部合并影响碎片形态。

表 10: CI 问题定位与处理

问题	原因	处理结果
格式检查失败	新增 C++ 代码未完全符合项目格式	运行格式化并重新提交
runner 磁盘不足	GitHub Actions 环境空间不足，影响 Go store binding 测试	增加 CI 磁盘清理步骤，避免无关失败
碎片化测试失败	测试构造中空闲块与尾部空闲区域合并	调整为内部块释放构造，保证最大连续区域低于请求大小
最终验证	上述问题修复后重新触发 CI	PR 头提交 <code>0123fa1</code> 通过 26 个检查，1 个检查跳过

4.6 文档、benchmark 与复现材料

除了代码本身，PR 和比赛材料还补充了三类工程化内容。第一，在 Mooncake 官方文档中说明 `fragmentation_aware` 的适用场景和启用方式，避免评审只看到

代码而不知道何时使用。第二，将新策略加入 `allocation_strategy_bench.cpp`，让后续真实 Store benchmark 可以直接横向比较 `random`、`free_ratio_first` 和 `fragmentation_aware`。第三，在比赛材料仓库中提供独立复现实验和日志，用轻量仿真固定问题模型，证明改进来自“排序信号变化”，而不是扩大候选规模或修改实验口径。

5 验证与效果

5.1 CI 验证

Mooncake 上游 draft PR 状态如表11所示。

表 11: PR CI 状态

项目	内容
PR 链接	https://github.com/kvcache-ai/Mooncake/pull/2797
头提交	0123fa1 Fix fragmentation-aware allocation test setup
CI 结果	26 个检查成功，1 个检查跳过
结论	上游构建、格式检查、文档构建和相关测试已通过。

5.2 专题仿真指标

专题仿真使用 5 个 segment 和 6 个大对象请求，模拟高总空闲但碎片化、低总空闲但连续空间充足两类 segment 并存的 Store 状态。结果如表12所示。

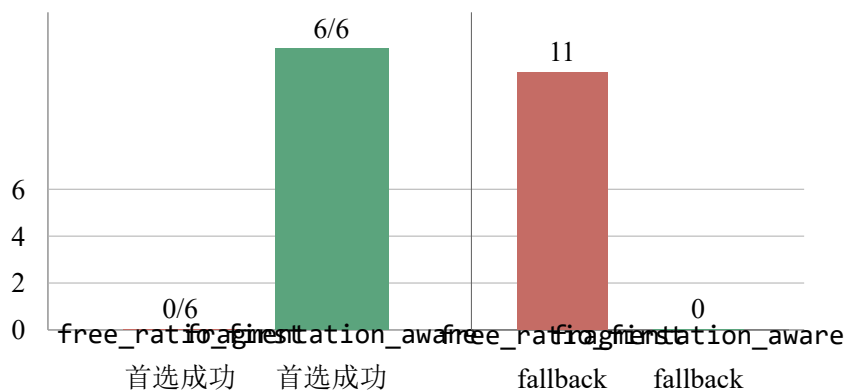


图 4: 首选成功次数与 fallback 尝试次数对比

表 12: 专题对齐仿真结果

指标	free_ratio_first	fragmentation_aware
大对象首选 segment 可直接容纳	0/6	6/6
最终可容纳	6/6	6/6
fallback 尝试次数	11	0
平均候选 segment 数量	5.00	5.00
单次决策耗时	121.00ns	138.93ns
新增决策开销	不适用	17.93ns

5.3 结果解释

该结果有三个含义。第一，`fragmentation_aware` 显著改善首选 segment 质量，在所有测试请求上都能首先选中可容纳 segment。第二，平均候选 segment 数量没有增加，说明改进来自排序信号变化，而不是扩大搜索范围。第三，单次决策新增开销约 17.93ns，在本地 CPU 仿真中处于很小量级，符合“用少量元数据计算换取更稳定分配路径”的设计目标。

6 与赛题要求的对应

表 13: 赛题要求映射

赛题关注点	本项目实现	当前证据
Mooncake Store 吞吐性能	减少碎片化导致的无效首选分配和 fallback 压力	专题仿真中 fallback 尝试次数从 11 降为 0
可扩展性	保持有界候选采样，不做全局扫描	平均候选 segment 数量保持 5.00
高可用语义	不改变副本 best-effort 路径，保留 preferred/excluded 语义	单元测试和策略设计说明
SGLang HiCache+Store	优化 Store 后端对象放置路径，为 HiCache 后端稳定性提供基础	方案定位和代码路径位于 Mooncake Store

开源规范	以 PR 形式提交上游 Mooncake，并保留 patch 和文档	PR#2797 通过 CI
可复现性	提供仿真源码、日志、技术报告和使用说明	repro/、logs/、EVALUATION.md

7 使用与复现

7.1 启用策略

```
mooncake_master --allocation_strategy=fragmentation_aware
```

7.2 应用补丁

```
git apply --check mooncake_fragmentation_aware_pr_2797_0123fa1.patch
git apply mooncake_fragmentation_aware_pr_2797_0123fa1.patch
```

7.3 回滚补丁

```
git apply -R mooncake_fragmentation_aware_pr_2797_0123fa1.patch
```

7.4 复现实验

```
g++ -std=c++17 -O2 repro/fragmentation_aware_sim.cpp -o
    fragmentation_aware_sim
./fragmentation_aware_sim

g++ -std=c++17 -O2 repro/fragmentation_aware_metrics.cpp -o
    fragmentation_aware_metrics
./fragmentation_aware_metrics

g++ -std=c++17 -O2 repro/topic_aligned_store_scalability_sim.cpp -o
    topic_aligned_store_scalability_sim
./topic_aligned_store_scalability_sim
```

8 边界与后续工作

本项目当前阶段已经完成代码实现、上游PR、CI验证、仿真评估和中文提交材料整理，但仍有边界需要明确：

- 当前不宣称已经完成真实RDMA硬件benchmark。
- 当前不宣称已经完成真实SGLang HiCache端到端吞吐压测。
- 当前不宣称已经重设计Mooncake高可用协议。
- 当前专题仿真验证的是分配策略层，不等同于完整生产集群压测。

后续如果进入决赛，建议补充三类工作：第一，在真实Mooncake Store环境中运行allocation_strategy_bench，统计吞吐、P50/P99、失败分配次数、fallback次数和内存利用率；第二，在具备RDMA条件的集群中验证跨节点场景；第三，将SGLang HiCache接入Mooncake Store后端，使用真实推理负载验证KVCache复用和后端IO表现。

9 结论

本文围绕Mooncake赛题2提出并实现Mooncake Store碎片感知分配策略。该策略针对free_ratio_first只看总空闲比例的不足，引入最大连续空闲区域和可容纳性排序，在不改变默认行为、不扩大候选规模、不破坏原有副本语义的前提下，提升碎片化场景下的大对象首选分配质量。当前PR已经通过上游GitHub Actions，确定性仿真显示首选成功次数从0/6提升到6/6，fallback尝试次数从11降到0。该工作是一个边界清晰、可复现、可回滚、适合继续扩展到真实Store benchmark和SGLang HiCache端到端测试的初赛成果。

A 提交材料清单

文件或目录	说明
README.md	项目入口说明
SUBMISSION.md	平台提交说明
DESIGN.md	设计文档
technical_solution.md	技术方案说明

EVALUATION.md	评估报告
testing.md	测试说明
usage.md	使用说明
REVIEW_GUIDE.md	评审阅读指南
mooncake_fragmentation_aware pr_2797_0123fa1.patch	当前PR补丁
repro/	复现实验源码
logs/	验证日志
slides/	初赛展示PPT
report/	本技术报告源码和PDF
release/	提交压缩包和校验文件

B 参考链接

- Mooncake 上游仓库: [kvcache-ai/Mooncake](#)
- 本项目 Mooncake PR: [kvcache-ai/Mooncake#2797](#)
- 比赛材料仓库: [Lorry1024/ccf2026-mooncake-fragmentation-aware](#)
- Mooncake fork 分支: [Lorry1024/Mooncake:ccf-fragmentation-aware-allocation](#)